

ASSIGNMENT ONE

Semester 2, 2026

ITAE6.100 Automation and Embedded Systems

Weighting:	70% of final grade
Length:	N/A
Due Date:	Refer to the schedule on the next page
Time:	5:00pm on the due date

The School of Computing has a policy of no late assignments. However, an assignment handed in up to 24 hours late will be marked. A deduction of 20% of the total marks available will be made. Any assignments received more than 24 hours late will not be marked but can be used as evidence of completing terms.

INSTRUCTIONS:

- You will work in **teams of two** and submit one set of lab sheets covering all four lab sessions.
- All submitted work must be original and entirely your own, except when incorporating ideas, quotations, tables, diagrams, code, or any other material from external sources. In such cases, proper acknowledgment must be given using the APA referencing style.
- Submitted work must not be reused for assessment in any other academic course.
- **For safety reasons, the assembly shall not be attached to mains (AC utility), a wall socket or any other potentially lethal power source.**

Instructions

Submission dates for Tasks

Completed tasks will be marked in an on-going process and any outstanding tasks marked after the completion date. Make a note at the top of the task, at the start of each session, of any absent team member. Absent team members will forfeit the task mark unless they have a valid reason. Lab work must be submitted by the specified date and time.

Lab activity	Topic	Number of sessions covered by lab activity	Expected Completion	Mark
1	Digital logic (Topics 1 & 2)	4	End of week 4 – 10 th August	25
2	Control systems (Topics 3 & 4)	4	End of week 8 – 7 th September	25
3	Programming with a PLC (Topics 5 & 6)	4	End of week 12 – 19 th October	25
4	HMI (Topic 7)	2	End of week 14 – 2 nd November	15
Total				90

Details

Each of the four tasks requires you conduct design, write, build software and hardware within various subject areas. Upon doing the investigations you will also be required to put various tasks into action where the task may be installing and configuring software or/and creating a circuit.

Marking

Marking guidance has been included at the end of this assessment document that can also be used as a checklist for each of the sections.

Team Member Contribution

Seeing as this is a team-based assessment it is important to assess the level of contribution of each team member to ensure fairness.

At the completion of the assessment, each team member is expected to submit a brief summary of their contribution and how they think their team member contributed/performed.

Use the following as a guide for the degree of contribution;

Criteria

- **Preparation** (Was the team member prepared for each of the lab activities?)
- **Participation** (Did each team member do their fair share of the lab activities?)
- **Reliability** (Was the team member reliable in delivering the work for each of the lab activities?)

Rating

1 = consistently poor

2 = usually poor

...

5 = ok

...

9 = usually excellent

10 = consistently excellent

Case Study: Simple Boiler Smart Controller

Section A. Assessment Introduction

The assessment for this course involves designing and implementing an automated system to control a simple water boiler.

The assessment is structured according to the topics covered in this course, which means we will follow the following approach:

- Design the solution using digital logic in Logisim (Topics 1 & 2)
- Solution implemented as program code on a microcontroller (Topics 3 & 4)
- PLC solution created in OpenPLC v3 (Topics 5 & 6)
- HMI solution with a PLC design (Topic 7)

Section B. Hardware Details of the Boiler Smart Controller

The logic and process control to drive the hot water is based on a Raspberry Pi microcontroller (RP2040, RP2350) device with peripheral devices. The boiler controller controls a power switch for the heater set by user driven choices. The user can select the desired heat and the Boiler Controller will heat the water to the set temperature. There are several indicators, buttons and fail-safe measures. A block diagram of the proposed boiler controller is provided below.

NOTE: The model may have missing features, controls, functions, etc.

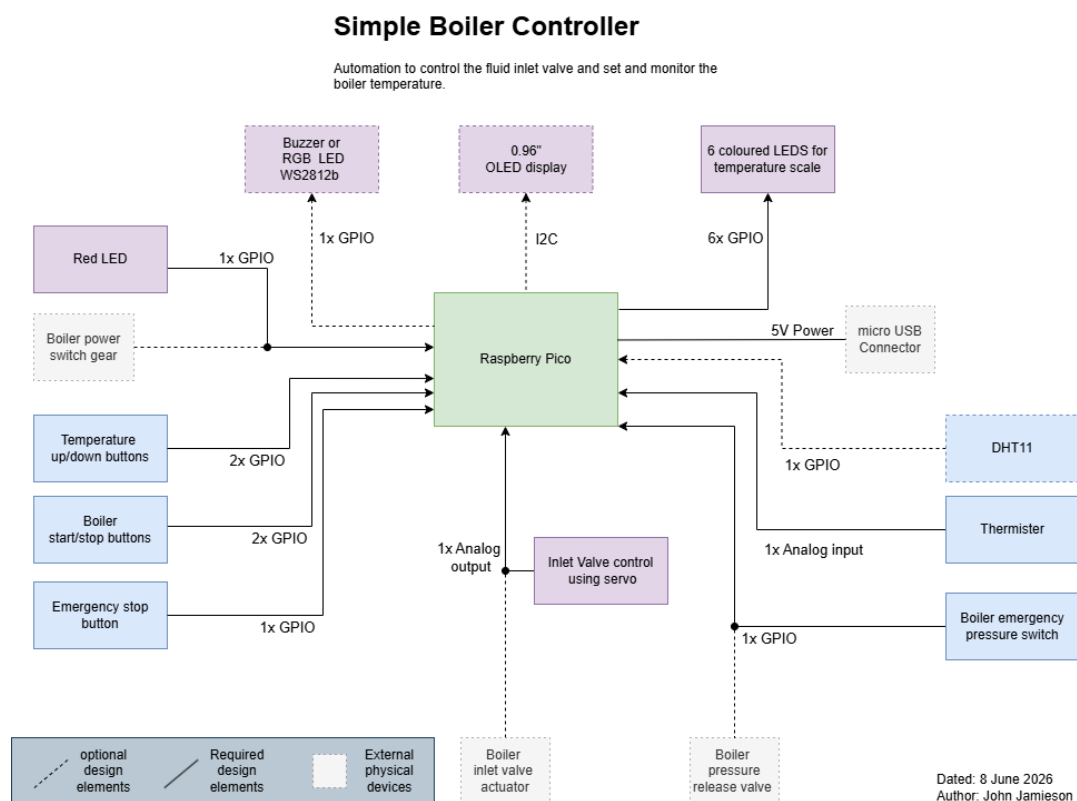


Figure 1 Diagram Block of the Simple Boiler Controller

Inputs & Outputs

The centre of the Boiler controller is the Raspberry Pico device. Attach to the Pico are the following interface controls and indicators.

Digital inputs

- Two buttons attached to digital inputs to start and stop the heating process
- Two buttons attached to digital inputs to adjust the temperature up and down from 80°C up to 180°C
- One button attached to a digital input for the emergency stop
- One digital input is used for an over pressure release switch

Digital outputs

- One digital output for a LED to indicate the boiler heater is activated
- Six digital outputs attached to 6 LEDs to indicate the temperature scale

Analogue Input/output

- One analogue output is used to driver a servo that would operate a water valve
- One analogue input for the temperature sensor (Thermistor). This will be the boiler temperature sensing component

Optional Inputs/Outputs

Peripherals displayed with a dotted line are not required to be implemented for the purposes of the case study. These provide additional features for example:

- **OLED display** – ideal to display the settings an operation status of the Boiler controller. This is deal for implementation in CircuitPython and possibly OpenPLC with Arduino extensions.
- **DHT11** – this if for the environmental temperature and humidity readings. This is deal for implementation in CircuitPython and possibly OpenPLC with Arduino extensions.
- **Buzzer** – this can be used for audible feedback to drive an alarm when the Boiler pressure switch is triggered. Can be implemented as Logic gates, CircuitPython or in OpenPLC.
- **RGB LED** – extra visual indicated to show temperature approaching setpoint or alarming, etc. This is a deal for implementation in CircuitPython and possibly OpenPLC with Arduino extensions.

All of the external devices (grey dotted boxes) are **NOT** implemented. They are present for the model completeness.

Implementation

A prototype of the device/appliance is to be created/assembled with either a **Raspberry Pico assembly** or the **picoPLC** or the **Waveshare relay board** and several components from the provided electronics kit. This device will be collectively referred to as the **PLC device** within this assessment. Only power provided through the USB port is permitted for the prototype assembly.

For safety reasons, the assembly shall not be attached to mains (AC utility), a wall socket or any other potentially lethal power source.

Section C. Operating procedure of the appliance

The goal is to duplicate the operation of the above appliance as a process control implementation for automation. The general operating behaviour to reproduce is described below. As you progress through the assessment you will be expected to design/create/duplicate the operation based on the requirements of the task.

Suggest Boiler Controller panel UI

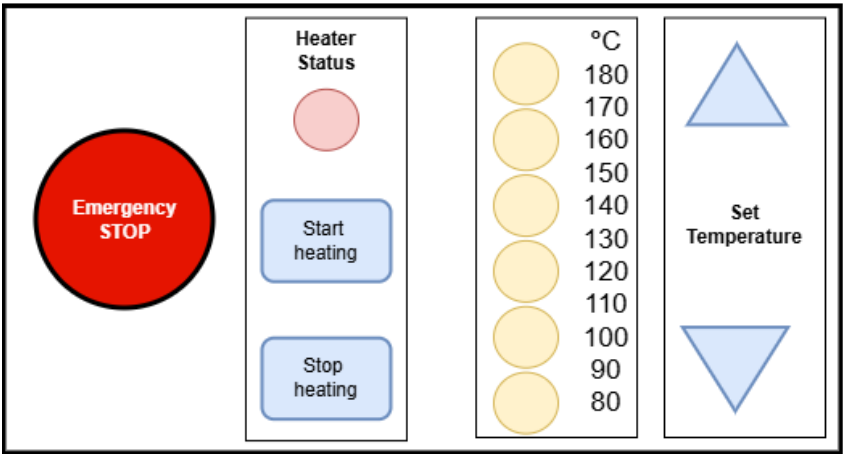


Figure 2 Basic UI for the boiler controller

Start/stop button operations

- When the **start button** (RUN_STATE) is pressed, the temperature set buttons are disabled. The heater is turned on (the red LED is turned on). The **emergency button** will respond to presses. The temperature set LEDs switch to become the temperature **status** LEDs.
- When the **stop button** (STOPPED_STATE) is pressed, the heater is turn off (the red LED is turned off). The emergency button presses are is ignored and the temperature set buttons are enabled. The temperature set LEDs switch to become the temperature **set** LEDs.

Temperature

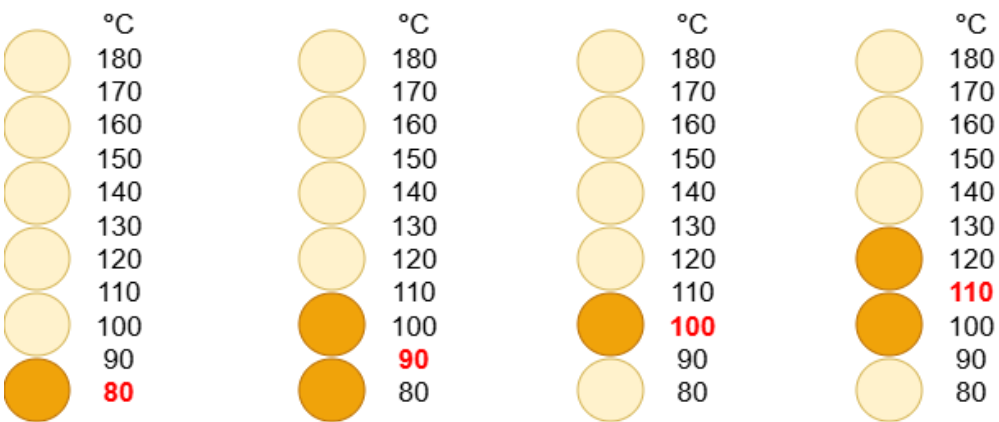


Figure 3 Temperature indicator

To represent the 11 temperature values, 6 LEDs are used where the in-between temperature values, 90, 110, 130, 150 and 170, are represented by turning on two LEDs for that “middle” temperature. The single LEDs,

when turned on, are then used to represent the 80, 100, 120, 140, 160 and 180 temperatures as shown in the Figure 3 above.

- The six LEDs are used to indicate the 11 temperature values within the 80-180 range as shown in the above.
- When the controller is in the STOPPED_STATE then the LEDs will indicate the desired (SETPOINT) temperature.
- When the controller is in the RUN_STATE then the LEDs will indicate the measured (ACTUAL_TEMPERATURE) temperature from the thermistor.

Pressure switch and emergency switch

- If the **pressure switch** is activated and the controller is in the RUN_STATE then the controller changes the operating state STOPPED_STATE. The switch is ignored in the STOPPED_STATE.
- If the **emergency switch** is activated and the controller is in the RUN_STATE then the controller changes the operating state STOPPED_STATE. The switch is ignored in the STOPPED_STATE.

Inlet valve control

- Water entering the boiler is controlled by a servo that slowly opens and closes the valve. When entering the RUN_STATE the servo should open from 0% to 80% (hint: duty cycle) in increments of 5%.
- When entering the STOPPED_STATE, the servo must close the valve by returning to 0% from the controller 80% in increments of 5%.

PID controller

A simple software PID controller is used to maintain the temperature based on the SETPOINT and the ACTUAL_TEMPERATURE.

Introduction

The focus of this part of the assessment is various aspects of digital logic and is based on topics discussed in class. The goal is to duplicate **some** of the case study operation as digital logic. The logic design activity requires the use of the (Logisim evolved¹) logic simulation software. The appliance has several digital inputs and outputs.

Requirements

- The operation of the boiler. E.g: How does the boiler know when it has enough water?
- NOTE: You are not required to implement missing features or controls.

Task: Button & light operation

- Create a logic circuit that reflects the operation of the buttons.
- Extend the above logic circuit to include the LED outputs.
- Include a counter to implement the temperature LED operation.
- No analogue circuits are required.

[25 marks]

Submission requirements for Part 1

For **Part 1 submission**, you need to document your work by taking **screenshots** of each step and providing a detailed explanation of the **logic** behind your Logisim design. Additionally, include a description of the **contributions** made by each team member. This should be compiled into a **Word document** named **<yourname>-part1.docx**. Along with the document, ensure that all **necessary Logisim files (.circ)** are included as evidence of your work. Once all required files are collected, place them in a folder and **compress it into a single ZIP file** named **Group*_Part1.zip**, where "*" represents your group number. This ZIP file will be your final submission.

Preparation for the next activity

To prepare for the next lab activity, ensure you have at least the following components and parts ready for assembly:

- PLC device ready for use.
- At least **seven** LED's (colours not important for the task) and resistors
- **Six** buttons
- **One** thermistor/temperature sensor (DHT11)
- **One** servo motor

¹ <https://github.com/logisim-evolution/logisim-evolution/>

Introduction

The focus of this part of the assessment is on leveraging a Microcontroller to implement a control system, and is based on topics covered in class. The objective of this assessment is to demonstrate your ability to use a microcontroller to achieve a set of requirements. Parts of the assessment might require additional research or actions to be able to move forward, e.g. installing or configuring software and reading datasheets.

This part of the assessment will continue to use the *Case Study Simple Boiler Smart controller*.

Details & Requirements

This assessment requires you to set up and configure the PLC device with suitable [CircuitPython](#) or [MicroPython](#) firmware and suitable editor (Thonny (preferred)² or [rvcircuit-studio](#)³) to demonstrate functioning programs using physical hardware. The course assumes CircuitPython but MicroPython is also possible, however you will need to make changes to the starter kit code if you choose to use MicroPython.

Some of the CircuitPython resources required for this task can be found in the **picoPLC starterkit**. You will find the following files that will be used in this activity:

- **Pico-OpenPLC-A4-Pinout.pdf** or **RP2350B_Relay_board_GPIO.xlsx** - pinout and GPIO mapping.
- **adafruit-circuitpython-raspberry_*.uf2** – CircuitPython firmware for the Raspberry Pico based boards.
- **adafruit-circuitpython-waveshare_*.uf2** - CircuitPython firmware for the Waveshare based board.
- **code** folder – contains sample code and the required **boot.py** & **iomapping*.py** file and **lib** folder for this activity. (**waveshare.py** and **raspberrypi.py** provide an OOP PLC wrapper to the GPIO)

To confirm operation on the breadboard, you must take **photos/video** of the “appliance” in action

Task 1: Process control in CircuitPython or MicroPython

With an understanding of how the case study appliance operates, the next step is to convert the operation into program code. The program will be written in CircuitPython or MicroPython.

- Install the required files and folders onto the Raspberry Pi Pico
- Map out the case study appliance inputs and outputs to the pinouts as reflected in the **Pico-OpenPLC-A4-Pinout.pdf** or **RP2350B_Relay_board_GPIO.xlsx** document and the **iomapping.py** file. E.g.
Temperature sensor – %IW0 (analogue input)
Water Value actuator - %QW (analogue output)
- Connect the necessary buttons, LEDs, servo and Temperature sensor to the PLC device (ensure the device is not plugged into a computer when setting it up).
 - Use the previous mapping to connect the components.
 - Use suitable resistors for the LEDs.
 - The inputs have an internal pulldown so buttons need to be connected to **3V3** and the GPIO for the Raspberry Pico based boards.
 - The buttons need to be connected to **DGND** and the GPIO for the Waveshare based board.
 - Use the **PCLcode1.py** and rename to **code.py** to test the inputs and outputs.
- Write a CircuitPython program that best reflects the operation of the case study appliance.

[20 marks]

² <https://thonny.org/>

³ <https://github.com/ArmstrongSubero/rvcircuit-studio/>

Task 2: Control system using a PID

Extend the above program code to include a suitable control system.

- Apply a PID controller to ensure the temperature control is maintained.
- Comment on why the PID in this scenario may not be the most suitable control system.

[5 marks]

Submission requirements for Part 2

For **Part 2 submission**, gather all required materials and place them in a folder named **Group*_Part2**, where "*" represents your group number. This folder should include **short videos (about 10 seconds each)** or **photos** demonstrating the hardware in action, along with **written documentation** explaining how you addressed challenges during the lab and detailing each team member's contributions. Additionally, include **all code files** for **Tasks 1 and 2**. Once everything is collected, **compress the folder into a single ZIP file** named **Group*-part2.zip** for final submission.

NOTE: Keep the assembly from the above circuit for the next lab activity

Introduction

The focus of this part of the assessment is on various aspects of Programmable Logic Controllers (PLCs) and is based on topics covered in class. Parts of the assessment might require additional research or actions to be able to move forward, *e.g.* installing or configuring software. In this task you will use three of the industry standard programming languages - Structured Text (ST), Ladder diagram (LD) and Function Block diagrams (FBD).

Details & Requirements

This part of the assessment will require you to set up and configure the OpenPLC PLC IDE to demonstrate functioning program using either simulation tool or their respective physical hardware. This part uses the PLC device with the **Pico-OpenPLC-A4-Pinout.pdf** or **RP2350B_Relay_board_GPIO.xlsx** mapping the pins as inputs and outputs. Make sure you are familiar with the pinout.

Make sure you have extracted and setup the **OpenPLC** distribution software. You will need the **OpenPLC * firmware.uf2** file from the **picoPLC_starterkit** to be installed onto the Raspberry Pi Pico. This WILL erase the CircuitPython installation so make sure you have a backup of your code.

This part of the assessment assumes you have retained the circuit from the previous lab activity.

To confirm operation on the PLC device, you must take screenshots/photos/video of the “appliance” in action

Task 1: IEC 61131-3 LD or FBD on the PLC device

- Use the OpenPLC application to create a **LD or FBD program** that meets the operation requirements for the appliance in the case study.
- Prepare to upload the program to the device, ensuring the “Enable Modbus RTU (Serial)” is checked. The baud should be 115200, the other settings can be left as default.
- Upload the program to the **PLC device** and confirm it is operating as intended.

[15 marks]

Save the program, from either the LD or FBD, as a ST file to be used with the OpenPLC Runtime in the next step

Task 2: OpenPLC runtime and the PLC device

- Once you have confirmed the above programs are working on Raspberry Pi Pico, the next step is to use the OpenPLC runtime to operate the Raspberry Pi Pico.
- Start the OpenPLC runtime and load the ST program generated from above.
- Confirm the OpenPLC runtime is correctly configured for the Raspberry Pi Pico.
- Start the Runtime and confirm the Raspberry Pi Pico is operating as intended.

NOTE: Not all of the PLC functionality will work in the OpenPLC Runtime without significant changes such as the PWM module.

[10 marks]

Submission requirements for Part 3

For **Part 3 submission**, collect all required materials and place them in a folder named **Group*_part3**, where "*" represents your group number. This folder should include your **Ladder Diagram (LD) or Function Block Diagram (FBD) program**, **short videos (around 10 seconds each)** demonstrating the correct functioning of your code for **Tasks 1 and 2**, and a **written document** explaining how you addressed challenges during the lab and outlining each team member's contributions. Once everything is gathered, **compress the folder into a single ZIP file** named **Group*_part3.zip** for final submission.

NOTE: Keep the above circuit & loaded program (LD or FBD) intact for the final lab activity

Introduction

The focus of this part of the assessment is on creating an HMI based on topics covered in class. Parts of the assessment might require additional research or actions to be able to move forward, *e.g.* installing or configuring software such as Python and supporting packages.

Details & Requirements

This part of the assessment will require create a simple UI written in Python and [NiceGUI](#) to demonstrate a functioning HMI program. This part uses the PLC device that has been preloaded with a program from the previous lab activity. Use the following figure as guidance for the interface. You can substitute image and other elements to represent the various UI widgets.

Suggest Boiler Controller panel UI

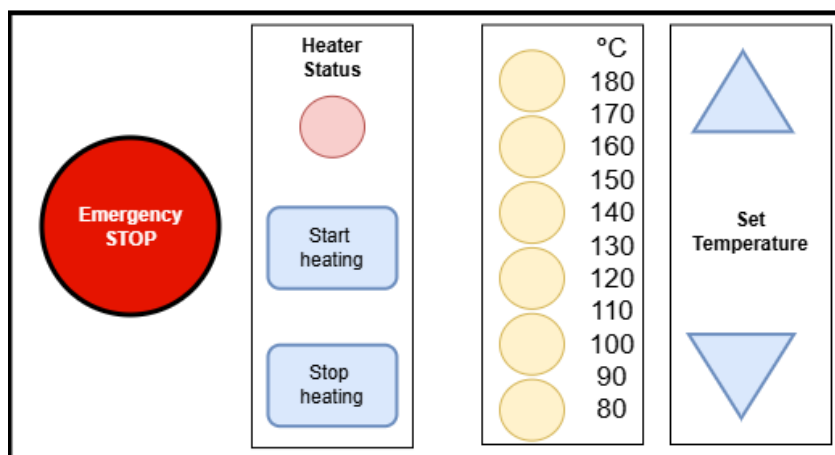


Figure 4 Basic UI for the boiler controller

The archive MUST include any screenshots, reports, documents you created too.

This part of the assessment assumes you have retained the breadboard circuit from the previous lab activity.

Portable Python kit

To simplify this task a portable Python installation has been prepared that includes the Python installation and supporting packages ([NiceGUI](#), [pyModbus](#), [PySerial](#)) for this project. Any additional packages are left up to your discretion and can be installed using the provided **pip.cmd** batch file. This installation has been prepared for a Microsoft Windows environment; however, you can use any other operating system too. Just install the above packages to test the demo code.

NiceGUI UI framework

The HMI makes use of the **NiceGUI** Python package that lets you write Python code to generate an interactive HTML page using your browser as the rendered. Various examples have been provided to assist you in creating

the above UI with only the basics of Python knowledge required. Refer to the [library documentation](#) for additional functionality.

Task: HMI UI for the Raspberry Pico PLC

- Create a UI that best reflects the requirements of the case study appliance in terms of the inputs and outputs.
- Connect the UI widgets the respective Modbus addresses used by the Raspberry Pi Pico program.

[15 marks]

Submission requirements for Part 4

For **Part 4 submission**, collect all required materials and place them in a folder named **Group*_Part4**, where "*" represents your group number. This folder should include **pictures and videos** related to the assessment, along with a **written document** explaining where and how you added or wrote the code during the lab. The document should also describe the **code function blocks** and detail each team member's contributions. Once all materials are organized, **compress the folder into a single ZIP file** named **Group*_Part4.zip** for final submission.

Checklist & marking guidance

Documentation and evidence included in the marking.

PART 1: DIGITAL LOGIC (Topics 1 & 2)

25 marks

Button & light operation

The logic circuit contains:

- Logic provision for button inputs [10 marks]
 - 5 buttons, 1 pressure switch
 - Inputs NOT required to be one-shot
- Logic provision for LED outputs. [10 marks]
 - 7 LED lights
- Logic operates in Logisim when various inputs activated

Scale marks accordingly

- Logic meets the requirement of the brief (18-20/20)
- Logic partially meets the requirement of the brief (10-17/20)
- Logic exists but does NOT meet the requirement of the brief (0-9/20)

[20 marks]

Counter operation

The logic circuit contains:

- Logic provision for 6 temperature LED outputs
- Counter with up to 6 stages
- Logic operates in Logisim when button inputs activated

Scale marks accordingly

- Logic meets the requirement of the brief (4-5/5)
- Logic partially meets the requirement of the brief (2.5-3/5)
- Logic exists but does NOT meet the requirement of the brief (0-2/5)

[5 marks]

For marking, we will assess two key aspects: **logic fulfilment** and **explanation**. You are expected to complete the logical implementation and provide a clear explanation to demonstrate your achievement of this week's learning outcome.

Content	Scale 1	Scale 2	Scale 3	Scale 4	Scale 5	Percentage	Mark out of 25
Logic fulfilment	Your logic circuit has the exact inputs and outputs, and it satisfies the operation of appliance. (19-20/20)	Your logic circuit has the exact inputs and outputs, and it overall satisfies the operation of appliance. (16-18/20)	Your logic circuit does not have exact number of inputs and outputs, but it overall satisfies the operation of appliance. (13-17/20)	Your logic circuit does not have exact number of inputs and outputs, but it only partly satisfies the operation of appliance. (10-12/20)	You do not provide any circuit evidence/Your logic circuit is not working. (0/20)	80%	0-20
Explanation	Your explanations show you have mastered the knowledge required in the course. (4-5/5)	Your explanations show you understand the knowledge to some extent but can be further improved. (3/5)	Your explanation is too simplified, and some key explanations are missing. (2/5)	Your explanation is not to the point. You should review and explain the process in detail. (1/5)	You did not provide any explanations on your logic circuit. (0/4)	20%	0-5

Process control in CircuitPython

The breadboard/picoPLC contains:

- Provision for 5 inputs (buttons or dupont wires)
 - Up/down, start/stop, estop
- 6 LEDS for temperature
- 1 LED or relay for heater output
- 1 analogue output for the servo

Scale marks accordingly for the circuit

- Circuit meets the requirement of the brief (8-10/10)
- Circuit partially meets the requirement of the brief (5-7/10)
- Circuit exists but does NOT meet the requirement of the brief (0-4/10)

Scale marks accordingly for the program

- Program meets the requirement of the brief (8-10/10)
- Program partially meets the requirement of the brief (5-7/10)
- Program exists but does NOT meet the requirement of the brief or work as intended (0-4/10)

[20 marks]

For marking, we will assess three key aspects: **Program code**, **System operation** and **explanation**. For system operation, you are expected to complete the assembly of hardware and program to make them work. For explanation, you should explain how you do your hardware assembly, your understanding of the code and what lessons were learnt in task 1.

Content	Scale 1	Scale 2	Scale 3	Scale 4	Scale 5	Percentage	Mark out of 20
CircuitPython Program	Your program meets all of the operating requirements of the case study. (8/8)	Most of the requirements operating requirements of the case study have been met. (6-7/8)	Some of the requirements operating requirements of the case study have been met. (5/8)	There is program code but it contains errors. (1-4/8)	You do not provide any circuit evidence/Your logic circuit is not working. (0/8)	40%	0-8
System operation (add Video, photos, etc.)	Well done! Your system consists of all components and they operate just according to the instructions of case study. (8/8)	Your circuit has most of the inputs and outputs, and it overall satisfies the operation of appliance. (6-7/8)	Your circuit is missing a number of inputs and outputs, but it partially satisfies the operation of appliance. (5/8)	Your circuit contains inputs/ outputs errors, an only partly satisfies the operation of appliance. (1-4/8)	You do not provide any circuit evidence/Your circuit is not working. (0/8)	40%	0-8
Explanation	Your explanations show you have mastered the knowledge required in the course. (4/4)	Your explanations show you understand the knowledge to some extent but can be further improved. (3/4)	Your explanation is too simplified, and some key explanations are missing. (2/4)	Your explanation is not to the point. You should review and explain the process in detail. (1/4)	You did not provide any explanation of your circuit. (0/4)	20%	0-4

Control system using a PID

- PID uses the DHT or LDR data
- PID drives the heater output based on temperature

Scale marks accordingly for the PID code

- PID functions as expected (4-5/5)
- PID function exists and functions but does not drive the output (2-3/5)
- PID exists but does not operate as intended (0-2/5)

[5 marks]

We will assess two aspects: **PID function implementation** and **explanation**. You should show clear evidence that the PID function code has been added to the system and is working properly. Additionally, explain how you integrated the PID function into the program and why PID may not be the best control system for this scenario. Your explanation should include comments in the code and demonstrate an understanding of PID's limitations or possible alternatives.

Content	Scale 1	Scale 2	Scale 3	Percentage	Mark out of 5
PID function (add Video, photos, etc.)	The PID function is added successfully in the system, and it is working. (3/3)	The PID function is added in the system but not working. (1-2/3)	You do not add PID function in the system. (0/3)	60%	0-3
Explanation	Your explanation is clear and to the point. (2/2)	Your explanation is not accurate, and you need to have another review on it. (1/2)	Your explanation is missing. (0/2)	40%	0-2

IEC 61131-3 LD on the Raspberry Pi Pico PLC

The ladder diagram/ function block diagram contains:

- Provision for 5 digital inputs and suitable variable names
- Provision for 7 digital outputs and suitable variable names.
- Provision for 1 analogue input and suitable variable names.
- Provision for 1 analogue output and suitable variable names.
- Logic operates according to the requirements brief

The program works in the simulator

The program uploads to the Raspberry Pi Pico

Scale marks accordingly for the LD/FBD

- Logic meets the requirement of the brief (13-15/15)
- Logic partially meets the requirement of the brief (8-12/15)
- Logic exists but does NOT meet the requirement of the brief (0-7/15)

Scale marks accordingly for the operation

- Program uploaded and Pico operates as intended (4-5/5)
- Program uploaded and Pico shows partial operation (2-3/5)
- Program uploaded but does not operate (0-1/5)

[20 marks]

We will assess two aspects: **LD or FBD on Pico PLC** and **Explanation**. For LD or FBD on Pico PLC, you should first show how you define your variables and logic of your program. Then you should run it in the OpenPLC IDE to prove its functionality. For explanation, we would like to see your ideas of how you program and how you use the OpenPLC.

Content	Scale 1	Scale 2	Scale 3	Scale 4	Scale 5	Percentage	Mark out of 15
LD or FBD on Pico PLC	Your LD/FBD program consist of enough number of variables and suitable variable names, and the code runs correctly in the OpenPLC simulator. According to your video and photo, the code also works perfectly with the Pico hardware. /Program uploaded, and Pico operates as intended. (12/12)	Your LD/FBD program consist of enough number of variables and suitable variable names, and the code runs correctly/partly in the OpenPLC simulator. According to your video and photo, the code only partly works with the Pico hardware. /Program uploaded, and Pico shows partial operation. (7-11/12)	Your LD/FBD program consist of suitable variable names, and the code runs correctly/partly in the OpenPLC simulator. According to your video and photo, you also fail to make it working on the hardware. /Program uploaded but does not operate. (6/12)	Your LD/FBD program do not consist of enough number of variables, and suitable variable names, and the code runs correctly/partly in the OpenPLC simulator. According to your video and photo, you also fail to make it working on the hardware. /Program uploaded but does not operate. (1-5/12)	You do not provide any LD/FBD evidence/Your LD/FBD is not working on PLC simulator and hardware. /Program cannot be uploaded and does not operate. (0/12)	80%	0-12

Explanation	Your explanations show you have achieved the learning outcome for this topic. (3/3)	Your explanations show you understand the knowledge to some extent but can be further improved. (2/3)	Your explanation is not to the point/Your explanation is too simplified, and some key explanations are missing. (1/3)	You do not provide any explanations on your logic circuit. (0/3)	Not applicable.	20%	0-3
-------------	---	---	---	--	-----------------	-----	-----

OpenPLC runtime and the Raspberry Pi Pico PLC

Scale marks accordingly for the Runtime operation

- Program uploaded to the Runtime (program running) and Pico operates as expected (4-5/5)
- Program uploaded to the Runtime (program running) and Pico is not functioning as expected (2-3/5)
- Program uploaded to the Runtime - program not running as expected (0-1/5)

[5 marks]

Building on Task 1, you are required to generate the ST file from OpenPLC IDE and use the OpenPLC runtime to operate the Raspberry Pi Pico.

We will assess two aspects: **OpenPLC Runtime** and **Explanation**. For OpenPLC runtime, you should show the step-by-step setting of the Runtime and working condition of Runtime. Runtime should be able to monitor the hardware. For explanation, we would like to see your lesson learnt from this task.

Content	Scale 1	Scale 2	Scale 3	Scale 4	Scale 5	Percentage	Mark out of 10
OpenPLC Runtime (add Video, photos, etc.)	Well done! The Runtime is set properly, and they operate just according to the instructions of case study. (8/8)	The Runtime is set properly and the monitor is partly working. (5-7/8)	The Runtime is set properly but the monitor is not/partly working. (4/8)	The Runtime setting is partly correct, and the monitor is not working (1-3/8)	You do not provide any evidence. (0/8)	80%	0-8
Explanation	Your explanations show you have mastered the knowledge required in the course. (2/2)	Your explanations can be further improved. /Your explanation is too simplified, and some key explanations are missing. (1/2)	There is no explanation provided. (0/2)	Not applicable	Not applicable	20%	0-2

HMI UI for the Raspberry Pico PLC

The HMI program contains and operates:

- The HMI aligns with the prototype guidance as per the task
- UI includes provision for
 - 7 LEDS
 - 5 Buttons

Scale marks accordingly for the SCADA operation

- The HMI mostly aligns with the requirement of the brief (8-10/10)
- The HMI partially aligns with the requirement of the brief with missing elements (5-7/10)
- The HMI exists but does NOT meet the requirement of the brief (0-4/10)

[10 marks]

Your created UI should best reflect the requirements of the case study appliance in terms of the inputs and outputs. The UI conditions must be synchronized according to the changing conditions of hardware (that is digital twin!)

We will assess on two key aspects: **UI operation** and **explanation**. For UI operation, we will evaluate your designed UI, its functionality, and how it interacts with the hardware. For explanation, you should describe your UI design process, demonstrate your understanding of the code, and reflect on the lessons learned from this task.

Content	Scale 1	Scale 2	Scale 3	Scale 4	Scale 5	Percentage	Mark out of 15
UI operation	Your designed UI contains all components, and the UI is designed beautifully. According to your video and photo, the UI works perfectly with the hardware. (12/12)	Your logic circuit has the exact inputs and outputs, and it overall satisfies the operation of appliance/partially operates. (7-11/12)	Your logic circuit does not have exact number of inputs and outputs, but it overall satisfies the operation of appliance. (6/12)	Your logic circuit does not have exact number of inputs and outputs, it partly satisfies the operation of appliance/does not operate. (1-5/12)	You do not provide any circuit evidence/Your logic circuit is not complete. (0/12)	80%	0-7
Explanation	Your explanations show you have mastered the knowledge required in the course. (3/3)	Your explanations show you understand the knowledge to some extent but can be further improved. (2/3)	Your explanation is not to the point/Your explanation is too simplified, and some key explanations are missing. (1/3)	You do not provide any explanations on your logic circuit. (0/3)	No applicable.	20%	0-3

Modbus connectivity for Raspberry Pico PLC

The HMI program contains code to enable Modbus communication with the PLC program

Scale marks accordingly for the UI operation

- The HMI operates correctly according to the requirement of the brief (4-5/5)
- The HMI partially operates according to the requirement of the brief with some missing features (2-3/5)
- The HMI exists does NOT operate according to the requirement of the brief (0-1/5)